Arab Academy for Science, Technology and Maritime Transport

College of Computing and Information Technology

System Programming Project Report

Five-Phase Compiler for a Small C++-Style Language

Using an SLR Parser and Switch-Case Support

Course	System Programming
Selected syntax analyzer	LR parser (bottom-up), specifically SLR(1)
Selected statement	switch - case - default with break
Implementation language	Python 3
Prepared by	Fill in group member names and IDs
Supervisor / TA	Fill in if required
Submission week	Week 11

April 22, 2026

Contents

1 Abstract

This report documents the design and implementation of a mini compiler developed for the System Programming project. The compiler implements the five required phases: context free grammar definition, lexical analysis, syntax analysis, semantic analysis, and optimized intermediate code generation using three-address code (TAC). The selected project combination is an LR parser for the `switch-case-default` statement with support for variable declarations and arithmetic expressions. The final system is implemented in Python and provides both command-line and graphical interfaces. In addition to the required functionality, the compiler also supports a small C++-style surface syntax with optional `#include`, optional `using namespace std;`, and restricted `cin`/`cout` statements. The implementation was validated using automated tests and sample programs covering valid syntax, syntax errors, semantic errors, and code generation.

2 Introduction

The main objective of the project is to build a complete compiler pipeline for a restricted programming language. According to the project instructions, the compiler must include:

1. a context free grammar,
2. a lexical analyzer,
3. a syntax analyzer,
4. a semantic analyzer,
5. optimized intermediate code generation using three-address code.

The selected project in this implementation is:

1. **Syntax analyzer method:** LR parser (bottom-up),
2. **Target statement:** `switch-case-default` with `break`.

To make the input programs feel familiar, the implementation accepts a small C++-style syntax. However, the core academic target remains the same: declarations, expressions, assignment statements, and the required `switch-case-default` construct. Loops and `if` statements are intentionally excluded because they are outside the chosen project scope.

3 Project Scope and Language Features

The supported language subset includes the following constructs:

1. optional `#include <...>` directives,
2. optional `using namespace std;`,
3. a single `int main() { ... return 0; }` entry function,
4. integer variable declarations,
5. integer assignments,
6. arithmetic expressions using `+`, `*`, and parentheses,
7. `switch`, `case`, `default`, and `break`,
8. restricted input/output using `cin`, `cout`, and `endl`.

The following features are not part of the selected project and are therefore not implemented:

1. `if` or `if-else`,
2. `while`, `do-while`, or `for` loops,
3. arrays,
4. user-defined functions other than `main`.

4 Overall System Architecture

The compiler is organized into small modules with clear responsibilities. This separation made the implementation easier to debug, test, and present.

Module	Responsibility
`grammar.py`	Defines productions, computes FIRST/FOLLOW sets, builds LR(0) states, and generates ACTION/GOTO tables.
`lexer.py`	Converts source code into tokens while removing whitespace and comments.
`parser.py`	Implements the SLR shift/reduce engine and builds the AST during reductions.
`semantic.py`	Performs declaration, duplication, compatibility, and switch-specific semantic checks. Also builds the symbol table.
`tac.py`	Generates three-address code instructions from the AST.
`optimizer.py`	Applies constant folding, constant propagation, unreachable code removal, and dead temporary elimination.
`pipeline.py`	Connects all compiler phases into one end-to-end workflow.
`models.py`	Holds shared dataclasses such as tokens, diagnostics, AST nodes, and TAC instructions.
`main.py`	Provides command-line entry and launches the GUI when needed.
`gui.py`	Presents the compiler phases visually and shows diagnostics, parse traces, AST, symbols, and TAC.
`runtime.py`	Additional feature: executes the restricted language to support the GUI console for `cin`/`cout`.

The execution order of the required phases is:

1. lexical analysis,
2. syntax analysis and AST construction,
3. semantic analysis,
4. TAC generation,
5. TAC optimization.

If an error appears in an earlier phase, the pipeline stops and does not continue to later phases. This prevents meaningless outputs such as TAC generated from an invalid AST.

5 Phase 1: Context Free Grammar Design

5.1 Design Goals

The grammar was designed to satisfy both the academic project requirements and the chosen language subset. The main goals were:

1. support the required `switch-case-default` structure,
2. support declarations and expressions as required by the project,
3. preserve operator precedence for arithmetic expressions,
4. remain suitable for bottom-up SLR parsing,
5. support optional C++-style wrappers without changing the core project statement.

5.2 Grammar Design Steps

The grammar was created through the following steps:

1. Define the highest-level program shape as optional include directives, optional namespace usage, and one `main` function.
2. Define `StmtList` as a sequence of statements.
3. Add declaration and assignment statements first because they are needed in all sample programs.
4. Add input and output statements as restricted stream operations.
5. Add the `switch` statement and allow each case to contain a statement list.
6. Add `break` as a standalone statement so the semantic phase can later validate its usage.
7. Build expression precedence using the standard hierarchy:
   (a) `Expr` for addition,
   (b) `Term` for multiplication,
   (c) `Factor` for identifiers, numbers, and parenthesized expressions.
8. Keep the grammar simple enough to remain conflict-free in SLR(1).

5.3 Complete Grammar

The final grammar used by the compiler is listed below.

```
Program -> OptIncludeList OptUsingDirective MainFunction
OptIncludeList -> IncludeList | epsilon
IncludeList -> IncludeList Include | Include
Include -> # include < id >
OptUsingDirective -> UsingDirective | epsilon
UsingDirective -> using namespace std ;
MainFunction -> int main ( ) { MainBody }
MainBody -> StmtList ReturnStmt | ReturnStmt
StmtList -> StmtList Stmt | Stmt
Stmt -> Decl | Assign | InputStmt | OutputStmt | SwitchStmt | BreakStmt
Decl -> int id ; | int id = Expr ;
Assign -> id = Expr ;
InputStmt -> InputRef InputList ;
InputRef -> cin | std :: cin
InputList -> InputList >> id | >> id
OutputStmt -> OutputRef OutputList ;
OutputRef -> cout | std :: cout
OutputList -> OutputList << OutputItem | << OutputItem
OutputItem -> Expr | string | EndlRef
EndlRef -> endl | std :: endl
Expr -> Expr + Term | Term
Term -> Term * Factor | Factor
Factor -> id | num | ( Expr )
SwitchStmt -> switch ( id ) { CaseList OptDefault }
CaseList -> CaseList Case | Case
Case -> case num : StmtList
OptDefault -> DefaultCase | epsilon
DefaultCase -> default : StmtList
BreakStmt -> break ;
ReturnStmt -> return num ;
```

5.4 Why This Grammar Fits LR Parsing

This grammar is appropriate for a bottom-up parser because:

1. left recursion is acceptable in LR parsing and is useful for expression lists,
2. operator precedence is encoded structurally,
3. case blocks are naturally handled as lists,

4. the grammar can be augmented and converted into an SLR parsing table.

The generated parser contains **107 LR(0) states** and is **conflict-free** in SLR(1), which confirms that the grammar is valid for the chosen parsing strategy.

6 Phase 2: Lexical Analysis

6.1 Objectives

The lexical analyzer reads raw source code and converts it into a stream of tokens. It also tracks line and column numbers to produce meaningful diagnostics.

6.2 Token Categories

The lexer recognizes the following token groups:

Category	**Examples**
Keywords	`int`, `main`, `switch`, `case`, `default`, `break`, `return`, `using`, `namespace`, `std`, `cin`, `cout`, `endl`, `include`
Identifiers	variable names such as `x`, `y`, `choice`
Numbers	integer literals such as `0`, `14`, `200`
Strings	text literals such as `"x = "`
Single-character symbols	`#`, `<`, `>`, `+`, `*`, `=`, `;`, `:`, `{`, `}`, `(`, `)`
Multi-character symbols	`::`, `«`, `»`

6.3 Design Steps

The lexical analysis phase was designed as follows:

1. Read the source program character by character.
2. Skip whitespace characters such as spaces, tabs, carriage returns, and newlines.
3. Detect and remove line comments starting with `//`.
4. Detect and remove block comments delimited by `/*` and `*/`.
5. Match multi-character operators before single-character symbols.
6. Read string literals while supporting escaped characters.
7. Recognize identifiers and decide whether they are keywords.

8. Recognize numeric literals.
9. Emit an error diagnostic for any unexpected character.
10. Append an end-of-file token `$`.

6.4 Representative Lexer Output

For the sample input in Appendix A, the beginning of the token stream is:

```
USING using
NAMESPACE namespace
STD std
SEMI ;
INT int
MAIN main
LPAREN (
RPAREN )
LBRACE {
INT int
ID x
ASSIGN =
NUM 2
PLUS +
```

This phase satisfies the project requirement that each input line must be broken into tokens after removing whitespace and comments.

7 Phase 3: Syntax Analysis and AST Construction

7.1 Chosen Method

The selected syntax analyzer is a **bottom-up LR parser**. The implementation uses an **SLR(1)** parsing strategy generated directly from the grammar.

7.2 Design Steps

The parser was built in the following sequence:

1. Augment the grammar with a new start production.
2. Compute FIRST sets for all nonterminals.

3. Compute FOLLOW sets for all nonterminals.
4. Build the canonical collection of LR(0) item sets using `closure` and `goto`.
5. Use FOLLOW sets to create the SLR ACTION/GOTO parsing table.
6. Detect conflicts during table construction.
7. Run a shift/reduce engine on the token stream.
8. Execute semantic actions during reductions to build the AST.

7.3 Shift/Reduce Parsing Procedure

During parsing, the implementation maintains:

1. a state stack,
2. a symbol stack,
3. a value stack,
4. a parse trace for debugging and visualization.

For each step:

1. the current state and lookahead token are used to access the ACTION table,
2. a `shift` action pushes a new state and token,
3. a `reduce` action pops the right-hand side symbols, executes a semantic action, and applies a GOTO transition,
4. `accept` ends the parse successfully,
5. a missing action produces a syntax error message showing the unexpected token and expected continuations.

7.4 AST Generation

The abstract syntax tree is built during parser reductions rather than in a separate pass. This makes the syntax analyzer directly produce the representation required by later phases.

Important AST node types include:

1. `Program`,
2. `Declaration`,
3. `Assignment`,
4. `BinaryOp`,
5. `Input`,
6. `Output`,
7. `Switch`,
8. `Case`,
9. `Default`,
10. `Break`,
11. `Identifier`,
12. `Number`,
13. `String`,
14. `Endl`.

7.5 Example AST in JSON Format

The project instructions require AST generation in JSON format. A simplified excerpt from a valid program is shown below:

```
{
  "type": "Program",
  "children": [
    {
      "type": "Declaration",
      "value": "int",
      "children": [
        { "type": "Identifier", "value": "x" },
        {
          "type": "BinaryOp",
          "value": "+",
          "children": [
            { "type": "Number", "value": 2 },
            {
              "type": "BinaryOp",
```

```
              "value": "*",
              "children": [
                { "type": "Number", "value": 3 },
                { "type": "Number", "value": 4 }
              ]
            }
          ]
        }
      ]
    }
  ]
}
```

8 Phase 4: Semantic Analysis

8.1 Objectives

The semantic analyzer checks whether a syntactically valid program also obeys the language rules. This phase also generates the symbol table required by the project instructions.

8.2 Implemented Checks

The implementation performs the following required checks:

1. **Check if all used variables are declared.**
2. **Check for no repeated variables.**
3. **Check if all variables are compatible in expression.**
4. **Generate a symbol table.**

Additional semantic checks were also implemented:

1. `break` is only valid inside a switch case,
2. duplicate `case` values are rejected,
3. a warning is produced for case fall-through when a case does not end with `break`,
4. a warning is produced when a switch statement has no default case,
5. unqualified `cin`, `cout`, and `endl` require `using namespace std;`.

8.3 Design Steps

The semantic phase follows these steps:

1. Create the global scope.
2. Traverse the AST from top to bottom.
3. Insert declarations into the symbol table.
4. Reject duplicate declarations before inserting them.
5. Resolve identifier uses through the current scope chain.
6. Infer expression types recursively.
7. Validate assignments and declaration initializers.
8. For each switch statement, create nested scopes for cases and default.
9. Record all final symbol entries for presentation in the GUI and report.

8.4 Symbol Table Example

For the valid sample program in Appendix A, the symbol table contains:

Name	Type	Scope	Line
`x`	`int`	`global`	4
`y`	`int`	`global`	5

8.5 Examples of Semantic Errors

Examples detected by the analyzer include:

1. `int x; int x;` -> duplicate declaration,
2. `y = 3;` when `y` is undeclared -> undeclared variable,
3. using a variable of the wrong type in an arithmetic expression,
4. repeating `case 1:` twice in the same switch.

9 Phase 5: Intermediate Code Generation Using TAC

9.1 Objectives

The intermediate code generator converts the AST into a lower-level, machine-independent representation. The chosen representation is three-address code (TAC), which is simple to optimize and easy to explain academically.

9.2 Instruction Design

The TAC generator uses instruction forms such as:

1. `decl type var`,
2. `x = y`,
3. `t1 = a + b`,
4. `t2 = a * b`,
5. `if x == 1 goto L1`,
6. `goto L2`,
7. labels such as `Lcase1:`,
8. restricted I/O instructions such as `cin » x` and `cout « y`.

9.3 Design Steps

The TAC generation phase was designed as follows:

1. Visit each top-level statement in program order.
2. For declarations, emit a `decl` instruction.
3. For initialized declarations and assignments, recursively generate expression code.
4. For arithmetic expressions, create temporary variables to preserve evaluation order.
5. For switch statements, lower the construct into conditional jumps and labels.
6. For `break`, emit a jump to the end label of the current switch.
7. For output, emit one TAC instruction per output item.

9.4 Lowering the Switch Statement

The `switch` statement is translated into TAC using this strategy:

1. generate one label for every case,
2. generate one end label for the whole switch,
3. compare the switch variable with each case constant,
4. jump to the corresponding case label when a match occurs,
5. jump to the default label if no case matches,
6. emit case bodies in order,
7. turn each `break` into `goto end_label`.

9.5 Example TAC

For the valid sample program, the compiler produces the following TAC:

```
decl int x
t1 = 3 * 4
t2 = 2 + t1
x = t2
decl int y
y = 0
cout << "x = "
cout << x
cout << endl
if x == 1 goto Lcase2
if x == 2 goto Lcase3
goto Ldefault4
Lcase2:
t3 = x + 1
y = t3
goto Lend1
Lcase3:
t4 = y * 2
y = t4
goto Lend1
Ldefault4:
y = 0
Lend1:
cout << "y = "
```

```
cout << y
cout << endl
```

10 Phase 6: TAC Optimization

10.1 Optimization Goals

The project requires optimized intermediate code generation. After TAC is created, the compiler runs an optimizer to improve the instruction sequence without changing program meaning.

10.2 Implemented Optimizations

Three optimizations are currently applied:

1. **Constant folding** for arithmetic on known constants,
2. **Constant propagation** into later assignments and outputs,
3. **Unreachable code elimination** after unconditional jumps,
4. **Dead temporary elimination** when generated temporaries are never used later.

10.3 Design Steps

The optimization phase executes the following passes:

1. Make a copy of the original TAC.
2. Scan instructions from top to bottom and keep an environment of known constants.
3. Replace arithmetic on constant operands by a direct assignment.
4. Clear the environment at labels and jumps to remain conservative.
5. Remove instructions that appear after an unconditional `goto` and before the next label.
6. Traverse the final code backward to remove assignments to unused temporary variables.

10.4 Example Optimized TAC

For the sample program, optimization simplifies the beginning of the code to:

```
decl int x
x = 14
decl int y
y = 0
cout << "x = "
cout << 14
cout << endl
```

The optimizer also reports informative notes such as:

1. `Constant folded t1 = 3 * 4.`
2. `Constant folded t2 = 2 + 12.`
3. `Removed dead temporary assignment:  t2 = 14`

11 Pipeline Control and User Interfaces

11.1 Compiler Pipeline

The `pipeline.py` module coordinates the five required phases. It stores all artifacts in one structure containing:

1. tokens,
2. diagnostics,
3. parse trace,
4. AST,
5. semantic result,
6. TAC,
7. optimized TAC.

This structure makes the compiler easy to inspect from both the command line and the GUI.

11.2 Command-Line Interface

The command-line mode prints:

1. diagnostics,
2. token stream,
3. parse steps,
4. AST as JSON,
5. TAC,
6. optimized TAC.

The report example was generated using:

```
python3 main.py --cli samples/valid_switch_case.txt
```

11.3 Graphical User Interface

Although the GUI is not part of the minimum project requirement, it improves demonstration quality. The interface presents:

1. source editor with syntax highlighting,
2. diagnostics panel,
3. token stream table,
4. LR parsing table and trace,
5. AST tree and JSON view,
6. symbol table and semantic diagnostics,
7. TAC and optimized TAC views,
8. minimal console for `cin`/`cout` execution.

12 Testing and Validation

12.1 Testing Strategy

The project was validated using a mixture of sample programs and automated unit tests. The tests cover:

1. successful compilation of a valid program,
2. correct reporting of syntax errors,
3. correct reporting of semantic errors,
4. conflict-free parsing table generation,
5. namespace rules for `cin`/`cout`,
6. runtime interaction for `cin` in the GUI console model.

12.2 Automated Test Cases

The following test cases are implemented:

1. `test_valid_program_runs_full_pipeline`
2. `test_invalid_syntax_is_reported`
3. `test_semantic_errors_stop_code_generation`
4. `test_parse_table_is_conflict_free`
5. `test_unqualified_io_requires_using_namespace_std`
6. `test_qualified_io_is_valid_without_using_namespace_std`
7. `test_runtime_executor_handles_cin_cout_and_switch`

They can be executed using:

```
python3 -m unittest discover -s tests -v
```

At the time of preparing this report, all tests pass successfully.

12.3 Sample Files

The following sample files are included for demonstration:

1. `samples/valid_switch_case.txt`
2. `samples/cin_cout_switch.txt`
3. `samples/invalid_syntax.txt`
4. `samples/semantic_errors.txt`

13 Compliance with Project Requirements

Table 1 summarizes how the implementation satisfies the stated project requirements.

Requirement	**How it is satisfied**
Generate CFG for the selected statement including declarations and expressions	Implemented in `grammar.py`; grammar includes declarations, expressions, and `switch-case-default`.
Lexical analyzer removes whitespace/comments and produces tokens	Implemented in `lexer.py`.
Syntax analyzer builds AST in JSON format	Implemented in `parser.py` and `models.py`.
Semantic analyzer checks declarations, duplicates, type compatibility, and symbol table	Implemented in `semantic.py`.
Generate optimized TAC	Implemented in `tac.py` and `optimizer.py`.
Use one of the allowed syntax methods	Implemented as an LR parser (SLR bottom-up).

Table 1: Requirement compliance summary

14 Suggested Team Work Distribution

The project instructions divide the work among three members. The current code base can be studied and presented using the following mapping:

Member	Recommended files and responsibilities
Member A	Context free grammar generation and lexical analyzer: `grammar.py`, `lexer.py`, and the token/production models in `models.py`.
Member B	Syntax analyzer and semantic analyzer: `parser.py`, `semantic.py`, and AST-related models in `models.py`.
Member C	TAC generation and optimization: `tac.py`, `optimizer.py`, and pipeline integration in `pipeline.py`.

This distribution aligns well with the project requirement that the phases be divided across group members.

15 Conclusion

This project successfully implements a five-phase compiler for a restricted language centered on the required `switch-case-default` construct. The compiler includes a formally defined grammar, a lexer, an SLR parser, AST generation, semantic validation, symbol table generation, TAC emission, and TAC optimization. The implementation is modular, testable, and demonstrable through both CLI and GUI interfaces. In addition to satisfying the required project phases, the system provides useful extras such as C++-style syntax, parse-trace visualization, and a minimal runtime console for interactive demonstrations.

A Appendix A: Representative Valid Input Program

```
using namespace std;

int main() {
    int x = 2 + (3 * 4);
    int y = 0;

    cout << "x = " << x << endl;

    switch (x) {
    case 1:
        y = x + 1;
        break;
    case 2:
        y = y * 2;
        break;
    default:
        y = 0;
    }

    cout << "y = " << y << endl;
    return 0;
}
```

B Appendix B: Example Optimizer Notes

```
[INFO] optimizer: Constant folded t1 = 3 * 4.
[INFO] optimizer: Constant folded t2 = 2 + 12.
[INFO] optimizer: Propagated constant into assignment of x.
[INFO] optimizer: Propagated constant into output statement.
[INFO] optimizer: Removed dead temporary assignment: t2 = 14
[INFO] optimizer: Removed dead temporary assignment: t1 = 12
```

C Appendix C: Files Included in the Submission

The source code submission should include at minimum:

1. all Python source files,
2. the `samples` directory,

3. this report,

4. the PowerPoint presentation prepared separately.